



DEVELOPER GUIDE

주간결 코드 안내서

처음 보는 사람을 위한

이 코드가 어떻게 생겼고, 어디를 어떻게 고치면 되는지. Next.js를 처음 보는 사람을 기준으로 씁니다.

대상 저장소 github.com/iphokorea1-boop/weekly-gyeol

기준 커밋 20e1b7e (2026-08-19)

함께 볼 문서 『주간결 인수인계 문서』 — 왜 그렇게 만들었는지는 그쪽에 있습니다

0 시작하기 전에

0.1 이 문서의 목표

이 코드를 혼자서 고칠 수 있게 만드는 것이 목표입니다. 전부 이해할 필요는 없습니다. 어디를 열어야 하는지만 알면 됩니다.

총 3,800줄이지만 겁먹을 필요 없습니다. 실제로 자주 만지는 파일은 열 개 남짓이고, 나머지는 한 번 읽고 다시 볼 일이 거의 없습니다. 4장까지 읽으면 코드가 어떻게 연결되는지 감이 오고, 5장의 실습을 따라 하면 직접 수정할 수 있습니다.

0.2 알고 있으면 좋은 것

알아야 함	수준
HTML / CSS	태그와 클래스가 무엇인지 아는 정도
JavaScript	변수, 함수, 배열의 <code>map</code> , <code>filter</code> , <code>async/await</code>
React	컴포넌트가 함수이고 화면 조각을 돌려준다는 것, <code>useState</code>
TypeScript	몰라도 됩니다. <code>: string</code> 같은 표시는 "이 값은 문자열"이라는 뜻일 뿐입니다
SQL	몰라도 됩니다. Prisma가 대신 써 줍니다

모르는 게 나와도 일단 넘어가세요

프로그래밍은 전부 이해하고 시작하는 게 아니라, 고치면서 이해하게 됩니다. 이 문서는 그 순서에 맞춰 쓰여 있습니다.

1 개발 환경 준비

- 1 **Node.js 설치** — nodejs.org에서 LTS 버전을 받습니다. 터미널에서 `node -v` 가 버전을 출력하면 성공입니다.

- 2 **코드 받기**

```
git clone https://github.com/iphokorea1-boop/weekly-gyeol.git
cd weekly-gyeol
npm install
```

- 3 **데이터베이스 연결 정보 넣기** — `.env.example` 을 복사해 `.env` 를 만들고, 두 개의 값을 채웁니다.

```
cp .env.example .env
```

`DATABASE_URL` 은 앱이 쓰는 연결, `DATABASE_URL_UNPOOLED` 는 스키마를 바꿀 때만 쓰는 연결입니다. Neon에서 프로젝트를 만들면 둘 다 받을 수 있습니다.

- 4 **테이블 만들기**

```
npx prisma migrate deploy
npx prisma generate
```

- 5 **실행**

```
npm run dev
```

브라우저에서 `http://localhost:3000` 을 엽니다. 로그인 화면이 나오면 정상입니다. 가입해서 들어가 보세요.

가장 중요한 준비: 연습용 데이터베이스

실제 서비스가 쓰는 데이터베이스를 그대로 연결하지 마십시오. 로컬에서 실수로 지운 것이 실제 데이터에서도 지워집니다. 이 프로젝트에서 실제로 그렇게 데이터가 한 건 사라진 적이 있습니다.

Neon 대시보드에서 *Branch*를 하나 만들면 운영 데이터를 복사한 별도 데이터베이스가 생깁니다. 로컬 `.env` 는 그쪽을 보게 하십시오.

2 프로젝트 지도

어떤 파일을 열어야 할지 모를 때 이 표를 보십시오.

2.1 폴더 구조

```
weekly-gyeol/
├─ app/                # 화면과 서버 코드가 전부 여기
│  ├─ page.tsx         # "오늘" 화면 → 주소 /
│  ├─ week/page.tsx    # "주간" 화면 → 주소 /week
│  ├─ month/page.tsx   # "월간" 화면 → 주소 /month
│  ├─ year/page.tsx    # "연간" 화면 → 주소 /year
│  ├─ login/page.tsx   # 로그인 화면 → 주소 /login
│  ├─ layout.tsx       # 모든 화면을 감싸는 껍데기 (상단 바 등)
│  ├─ globals.css      # 색·애니메이션 등 전역 스타일
│  ├─ components/      # 화면 조각들 (재사용되는 부품)
│  ├─ api/             # 브라우저가 호출하는 서버 기능
│  └─ actions/         # 폼 제출을 서버에서 처리하는 함수
├─ lib/               # 화면과 무관한 순수 로직
│  ├─ task-utils.ts    # 날짜·시간 계산 (매우 중요)
│  ├─ gamification.ts  # 연속 기록·XP·배지 계산
│  ├─ holidays.ts     # 공휴일 계산 (DB를 쓰지 않음)
│  ├─ nudge.ts         # 활동 촉진 메일: 보낼지 판단 + 본문
│  ├─ shortcuts.ts     # 단축키 표 (키 해석 + 도움말이 함께 씀)
│  ├─ mailer.ts        # 메일 발송 (Resend REST API)
│  ├─ auth.ts          # 비밀번호와 세션
│  ├─ dal.ts           # "지금 로그인한 사람이 누구인가"
│  └─ prisma.ts        # 데이터베이스 연결
├─ prisma/
│  ├─ schema.prisma    # 데이터베이스 설계도
│  └─ migrations/     # 설계 변경 이력
└─ proxy.ts           # 모든 요청이 가장 먼저 지나가는 문지기
```

폴더 이름이 곧 주소입니다

Next.js의 App Router에서는 `app/week/page.tsx` 파일이 자동으로 `/week` 주소가 됩니다. 라우팅 설정 파일을 따로 만들지 않습니다. 새 화면을 만들려면 폴더를 만들고 그 안에 `page.tsx` 를 넣으면 끝입니다.

2.2 "이걸 고치려면 어디를 열어야 하나"

하고 싶은 일	열어야 할 파일
화면에 보이는 문구 수정	해당 <code>page.tsx</code> 또는 <code>components/</code> 의 파일
색 바꾸기	<code>app/globals.css</code> 맨 위 <code>:root</code>

애니메이션·누름 효과 조절	app/globals.css 의 <code>.pressable</code> , <code>.lift</code>
할 일 목록에 보이는 항목 모양	app/components/task-item.tsx
주간 격자 안의 일정 블록	app/components/task-block.tsx
"할 일 추가" 버튼 (오늘 페이지)	app/components/add-task-form.tsx
연속 기록·XP·배지 규칙	lib/gamification.ts
날짜 계산	lib/task-utils.ts
공휴일 추가·수정 (2031년 이후 등)	lib/holidays.ts 의 LUNAR 표
드래그로 옮기는 동작	app/components/drag-context.tsx (엔진), week-board.tsx · month-board.tsx (놓을 자리)
할 일 추가 폼의 칸·문구	app/components/task-form.tsx (오늘 페이지와 달력이 함께 씀)
날짜를 눌러 추가하는 동작	app/components/quick-add.tsx (팝오버), 각 보드의 addAtTime · addOnDay
단축키 추가·변경	lib/shortcuts.ts 의 SHORTCUTS 와 resolveShortcut (표에 넣으면 도움말에도 자동으로 나옵니다)
촉진 메일 문구·발송 조건	lib/nudge.ts
메일이 언제 나가는지	vercel.json 의 crons , app/api/cron/nudge/route.ts
데이터베이스에 컬럼 추가	prisma/schema.prisma (5.4 참고)
로그인 화면	app/components/auth-form.tsx

3 반드시 이해해야 할 다섯 가지

이 다섯 개만 알면 나머지는 읽으면서 따라올 수 있습니다.

3.1 서버에서 도는 코드와 브라우저에서 도는 코드

이 프로젝트에서 가장 헷갈리는 부분입니다. 같은 `.tsx` 파일처럼 보여도 실행되는 장소가 다릅니다.

	서버 컴포넌트 (기본값)	클라이언트 컴포넌트
표시	아무것도 안 씀	파일 맨 위에 <code>"use client"</code>
도는 곳	서버	사용자의 브라우저
할 수 있는 것	데이터베이스 조회, 비밀 값 사용	클릭 처리, <code>useState</code> , 애니메이션
할 수 없는 것	<code>onClick</code> , <code>useState</code>	DB 직접 조회 (반드시 서버를 거쳐야 함)
이 프로젝트의 예	<code>app/page.tsx</code> , <code>app/week/page.tsx</code>	<code>task-item.tsx</code> , <code>nav.tsx</code> , <code>auth-form.tsx</code>

왜 나누는가

서버 컴포넌트는 결과 HTML만 브라우저로 보냅니다. 데이터베이스 접속 정보 같은 비밀이 브라우저로 넘어가지 않고, 브라우저가 받는 코드 양도 줄어듭니다. 대신 클릭 같은 상호작용은 브라우저에서 일어나야 하므로, 상호작용이 필요한 조각만 클라이언트 컴포넌트로 만듭니다.

실제 구조를 보면 이렇습니다. `app/page.tsx` (서버)가 데이터베이스에서 할 일을 읽어 오고, 그것을 `TaskItem` (클라이언트)에 값으로 넘겨줍니다. 클릭은 `TaskItem` 이 처리합니다.

`app/page.tsx` - 서버에서 실행됨

```
export default async function Home() {
  const user = await requireUser(); // 로그인 확인
  const tasks = await prisma.task.findMany({ // DB 조회 - 서버라서 가능
    where: { archived: false, userId: user.id },
  });

  return tasks.map((t, i) => /* 아래 컴포넌트에 값으로 전달 */
    <TaskItem key={t.id} task={...} index={i} />);
}
```

자주 하는 실수

서버 컴포넌트에 `onClick` 을 쓰면 오류가 납니다. 반대로 클라이언트 컴포넌트에서 `prisma` 나 `lib/auth.ts` 를 import해도 오류가 납니다. 오류 메시지가 이 경계를 알려주므로 겁먹지 말고 읽으십시오.

3.2 Prisma — 데이터베이스를 JavaScript처럼

SQL을 직접 쓰지 않습니다. `prisma/schema.prisma` 에 표 모양을 적어 두면, Prisma가 그에 맞는 함수와 타입을 만들어 줍니다.

`prisma/schema.prisma`

```
model Task {
  id      String   @id @default(cuid())
  title   String
  dueDate DateTime? // ? 는 "없을 수도 있음"
  userId  String?
```

```
// 위 정의만 있으면 이런 코드를 쓸 수 있고, 오타는 타입 검사에서 잡힙니다.
await prisma.task.findMany({ where: { userId: user.id } });
await prisma.task.create({ data: { title: "운동하기" } });
await prisma.task.deleteMany({ where: { id, userId: user.id } });
```

주로 쓰는 함수는 여섯 개뿐입니다.

함수	하는 일
<code>findMany</code>	조건에 맞는 행 여러 개 가져오기
<code>findUnique</code>	유일한 값(id 등)으로 한 개 가져오기
<code>findFirst</code>	조건에 맞는 첫 한 개 가져오기
<code>create</code>	새로 만들기
<code>updateMany</code>	조건에 맞는 것 수정 (이 프로젝트에서 중요)
<code>deleteMany</code>	조건에 맞는 것 삭제 (이 프로젝트에서 중요)

데이터를 눈으로 보고 싶다면

`npx prisma studio` 를 실행하면 브라우저에서 표 형태로 데이터베이스를 직접 보고 수정할 수 있습니다. 처음 구조를 파악할 때 매우 유용합니다.

3.3 Tailwind — CSS를 클래스 이름으로

별도의 CSS 파일을 만들지 않고 클래스 이름으로 스타일을 씁니다. 처음에는 길어 보이지만 규칙은 단순합니다.

```
<div className="flex items-center gap-2.5 rounded-xl border py-2.5">
```

클래스	뜻
<code>flex</code>	가로로 나란히 배치
<code>items-center</code>	세로 가운데 정렬
<code>gap-2.5</code>	사이 간격 (숫자 × 4px = 10px)
<code>rounded-xl</code>	모서리 둥글게
<code>py-2.5</code>	위아래 안쪽 여백 (<code>p</code> =padding, <code>y</code> =세로)
<code>text-ink-soft</code>	이 프로젝트가 직접 정한 색 (아래 참고)
<code>hover:bg-surface-sunk</code>	마우스를 올렸을 때만 배경색 변경

`text-ink-soft`, `bg-dated-soft` 처럼 닷선 이름은 이 프로젝트가 `app/globals.css` 에서 직접 정의한 색입니다. 색을 바꾸고 싶다면 클래스가 아니라 그 파일의 `:root` 를 고치면 됩니다(실습 5.2).

3.4 Server Action — 폼을 서버에서 바로 처리

로그인 폼이 쓰는 방식입니다. `fetch` 로 API를 호출하는 대신, 서버 함수를 `<form action={...}>` 에 직접 연결합니다.

`app/actions/auth.ts`

```
"use server"; // ← 이 줄이 "이 파일은 서버에서만 돈다"는 표시

export async function login(_prev, formData: FormData) {
  const email = String(formData.get("email")).trim().toLowerCase();
  // ... 비밀번호 확인 후 세션 발급 ...
  redirect("/");
}
```

장점은 비밀번호가 브라우저 코드에 남지 않는다는 점입니다. API 주소를 만들 필요도 없습니다.

3.5 지금 로그인한 사람이 누구인가

로그인하면 브라우저에 `session` 쿠키가 저장됩니다. 이후 모든 요청에 자동으로 따라붙고, 서버는 그것으로 사용자를 알아냅니다. 그 변환은 **딱 한 곳**에서만 일어납니다.

lib/dal.ts

```
export const getCurrentUser = cache(async () => {
  const token = (await cookies()).get(SESSION_COOKIE)?.value;
  if (!token) return null;
  // 쿠키 값을 해시로 바꿔 DB에서 세션을 찾고, 만료를 확인한다
});

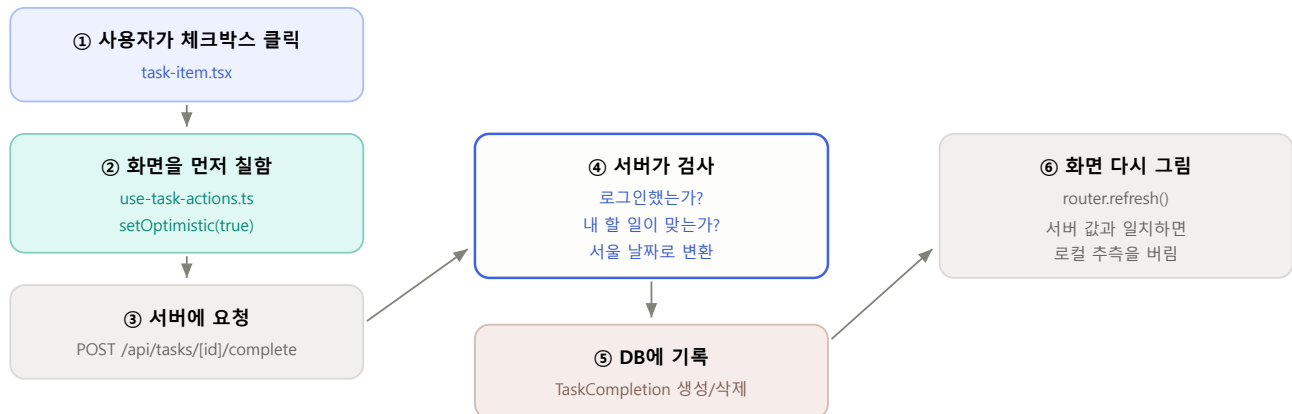
/** 페이지용: 로그인 안 했으면 /login으로 보낸다 */
export async function requireUser() { /* ... */ }
```

새 페이지를 만들 때 반드시

페이지 첫 줄에서 `await requireUser()` 를 부르고, 데이터를 조회할 때 `where` 에 `userId: user.id` 를 넣으십시오. **이걸 빠뜨리면 남의 데이터가 보입니다.** 화면에서 숨기는 것으로는 막을 수 없습니다.

4 따라가 보기: 체크박스를 누르면

코드가 어떻게 연결되는지 감을 잡는 가장 좋은 방법은 동작 하나를 끝까지 추적하는 것입니다.



핵심: ②가 ③보다 먼저다 — 그래서 네트워크가 느려도 즉시 반응한다

체크박스 한 번의 전체 경로

단계별 실제 코드

① 클릭을 받는 곳 — `app/components/task-item.tsx`

```
<button onClick={() => toggle(task.occurrenceDate ?? new Date(), task.xp)}>
```

② 즉시 칠하고 ③ 서버에 요청 — `app/components/use-task-actions.ts`

```
async function toggle(date, xp) {
  const next = !done;
  setOptimistic(next); // ← 화면 먼저

  const res = await fetch(`/api/tasks/${taskId}/complete`, {
    method: "POST",
    body: JSON.stringify({ date: date.toISOString() }),
  });
  // 서버가 다른 답을 주면 정정
}
```

④⑤ 서버가 검사하고 기록 — `app/api/tasks/[id]/complete/route.ts`

```
const user = await getCurrentUser();
if (!user) return NextResponse.json({ error: "unauthorized" }, { status: 401 });

// 내 할 일이 맞는지 먼저 확인 (TaskCompletion에는 주인 정보가 없으므로)
const owned = await prisma.task.findFirst({ where: { id, userId: user.id } });
if (!owned) return NextResponse.json({ error: "not found" }, { status: 404 });

const dateOnly = toDateOnly(body.date ? new Date(body.date) : new Date());
```

왜 서버에서 날짜를 다시 계산하나

브라우저가 보낸 것은 "정확한 순간"(예: 2026-08-18T16:00Z)입니다. 서버는 그것이 **서울 기준 며칠인지**를 다시 판단해야 합니다. 이 변환이 없으면 밤 늦게 체크한 기록이 전날로 저장됩니다.

5 직접 고쳐보기

쉬운 것부터 순서대로 되어 있습니다. 하나씩 따라 하면 어디를 어떻게 만지는지 몸에 익습니다. 각 실습 뒤에는 `npx tsc --noEmit` 으로 타입 오류가 없는지 확인하십시오.

5.1 문구 바꾸기 난이도 1

할 일이 하나도 없을 때 나오는 문구를 바꿔봅시다. `app/page.tsx` 에서 찾습니다.

```
<p className="rounded-lg border border-dashed ...">
  오늘 예정된 할 일이 없어요.      // ← 이 줄을 바꾸면 됩니다
</p>
```

`npm run dev` 가 켜져 있으면 저장하는 순간 브라우저에 반영됩니다.

5.2 색 바꾸기 난이도 1

색은 클래스가 아니라 `app/globals.css` 맨 위에서 한 번에 정의됩니다. 여기만 고치면 앱 전체가 따라 바뀝니다.

`app/globals.css`

```
:root {
  --dated:      #3e63dd; /* 날짜 있는 할 일 - 진한 색 */
  --dated-ink:  #3a5bc7; /* 글자에 쓰는 더 어두운 색 */
  --dated-soft: #edf2fe; /* 배경에 쓰는 아주 연한 색 */
  --dated-line: #abddf9; /* 테두리 */
}
```

다크 모드도 같이 고쳐야 합니다

같은 파일 아래쪽 `@media (prefers-color-scheme: dark)` 블록에 어두운 화면용 값이 따로 있습니다. 위쪽만 바꾸면 다크 모드에서 옛 색이 남습니다.

색을 고를 때는 **연한 배경 위의 글자색 대비**를 확인하십시오. Radix Colors(radix-ui.com/colors)에서 고르면 단계별로 대비가 맞춰져 있어 안전합니다.

5.3 배지 추가하기 난이도 2

배지는 저장되지 않고 **완료 기록에서 매번 계산**됩니다. 그래서 배열에 항목 하나를 더하는 것으로 끝납니다.

`lib/gamification.ts` - `computeAchievements` 안의 배열

```
{
  id: "night",
  name: "올빼미",
  hint: "밤 10시 이후에 5건 완료",
  earned: nightOwls >= 5,
  progress: bar(nightOwls, 5),
},
```

위 `nightOwls` 는 같은 함수 안에서 미리 세어 두어야 합니다. 기존 `earlyBirds` 계산 바로 아래에 추가하십시오.

```
const nightOwls = completions.filter(
  (c) => hourInSeoul(new Date(c.completedAt)) >= 22
).length;
```

시간을 다룰 때는 반드시 `hourInSeoul`

`new Date(...).getHours()` 를 쓰면 **서버 시간대(UTC)**의 시각이 나옵니다. 그러면 "밤 10시"가 한국 시간 아침 7시가 됩니다. 이 프로젝트에서 실제로 있었던 버그입니다.

5.4 데이터에 항목 추가하기 난이도 3

할 일에 장소를 넣어봅시다. 데이터베이스 구조가 바뀌므로 단계가 가장 많습니다. 순서를 지키십시오.

- 1 설계도 수정 — `prisma/schema.prisma` 의 `Task` 에 한 줄 추가.

```
place String? // ? 를 붙여 "없어도 됨"으로. 기존 행이 이미 있으므로 필수로 만들면 실패합니다
```

- 2 변경될 SQL을 먼저 눈으로 확인 — 적용하지 않고 내용만 봅니다.

```
npx prisma migrate diff \
  --from-url "$DATABASE_URL_UNPOOLED" \
  --to-schema-datamodel prisma/schema.prisma --script
```

`DROP` 이나 `DELETE` 가 보이면 **멈추고** 무엇이 잘못됐는지 확인하십시오. 여기서는 `ALTER TABLE ... ADD COLUMN` 만 나와야 합니다.

- 3 마이그레이션 파일로 저장하고 적용 —

`prisma/migrations/20260820120000_add_place/migration.sql` 로 저장한 뒤

```
npx prisma migrate deploy
npx prisma generate
```

- 4 저장 경로 열어주기 — `app/api/tasks/route.ts` 의 `POST` 에서 새 값을 받도록 합니다.

```
const { title, memo, dueDate, place, ... } = body;

const task = await prisma.task.create({
  data: { title: title.trim(), place: place || null, ..., userId: user.id },
});
```

- 5 **입력칸 만들기** — `app/components/task-form.tsx` 에 `useState` 와 `<input>` 을 추가하고, `submit` 의 `body` 에 `place` 를 넣습니다. 이 파일 하나만 고치면 오늘 페이지와 달력 팝오버에 **동시에** 나타납니다.
- 6 **화면에 보여주기** — `app/components/task-item.tsx` 의 `TaskItemData` 타입에 `place?: string | null` 을 더하고, `app/page.tsx` 의 변환 함수에서 값을 넘겨줍니다.

왜 이렇게 여러 곳을 고쳐야 하나

데이터가 **DB → 서버 → 화면** 순으로 흐르기 때문에, 중간 어디 하나라도 빠지면 값이 전달되지 않습니다. 다행히 TypeScript가 빠진 곳을 오류로 알려주므로 `npm tsc --noEmit` 을 실행하면서 하나씩 메우면 됩니다.

5.5 새 화면 만들기 난이도 3

`app/settings/page.tsx` 파일을 만들면 `/settings` 주소가 생깁니다. 아래가 최소 골격입니다.

`app/settings/page.tsx`

```
import { prisma } from "@/lib/prisma";
import { requireUser } from "@/lib/dal";

export const dynamic = "force-dynamic"; // 매번 새로 그림 (로그인 상태에 따라 달라지므로)

export default async function SettingsPage() {
  const user = await requireUser(); // ① 로그인 확인 - 절대 빼면 안 됨

  const count = await prisma.task.count({
    where: { userId: user.id }, // ② 내 것만 - 절대 빼면 안 됨
  });

  return (
    <div className="mx-auto w-full max-w-3xl px-4 pt-5">
      <h1 className="text-2xl font-extrabold">설정</h1>
      <p className="mt-2 text-sm text-ink-soft">{user.email} · 할 일 {count}건</p>
    </div>
  );
}
```

상단 탭에도 넣으려면 `app/components/nav.tsx` 의 `TABS` 배열에 항목을 추가하면 됩니다. 아이콘은 `lucide-react` 에서 가져옵니다.

6 절대 하면 안 되는 것

전부 이 프로젝트에서 실제로 문제가 됐던 것들입니다.

1. 운영 데이터베이스에 대고 테스트하기

자동화 테스트나 실험을 실제 데이터가 든 DB에 대고 돌리지 마십시오. 이 프로젝트에서 브라우저 자동화 도중 할 일 한 건이 삭제된 적이 있습니다. Neon 브랜치를 만들어 로컬은 그쪽을 보게 하십시오.

2. 날짜를 직접 계산하기

`new Date(2026, 7, 19)`, `getDate()`, `toLocaleDateString()` (시간대 미지정)을 쓰지 마십시오. 반드시 `lib/task-utils.ts` 의 함수를 쓰십시오.

쓰지 말 것	대신 쓸 것
<code>new Date()</code> 로 오늘 구하기	<code>todayInSeoul()</code>
<code>new Date(y, m, d)</code>	<code>makeDate(y, m, d)</code>
<code>d.getDay()</code>	<code>weekdayOf(d)</code>
<code>d.getHours()</code>	<code>hourInSeoul(d)</code>
<code>d.toLocaleDateString("ko-KR", ...)</code>	<code>formatKo(d, ...)</code>

3. 소유자 조건을 빠뜨리기

조회에는 `where: { userId: user.id }`, 수정·삭제에는 `updateMany / deleteMany` 에 소유자 조건을 넣으십시오. `update({ where: { id } })` 는 남의 데이터를 고칠 수 있는 취약점입니다. 화면에서 버튼을 숨기는 것은 방어가 아닙니다. 주소창으로 직접 호출할 수 있습니다.

4. 운영 DB에 `prisma migrate dev` 실행

스키마가 어긋나면 데이터베이스를 초기화하겠다고 제안합니다. 잘못 승인하면 전부 사라집니다. 운영에는 항상 `migrate deploy` 만 쓰고, 그 전에 `migrate diff` 로 SQL을 눈으로 확인하십시오.

5. `dark`: 클래스 쓰기

이 앱의 다크 모드는 `.dark` 클래스가 아니라 운영체제 설정(`prefers-color-scheme`)으로 동작합니다. 그래서 `dark:bg-black` 같은 클래스는 영원히 적용되지 않습니다. 다크 모드에서 다른 색을 쓰고 싶으면 `globals.css` 에 CSS 변수를 만들어 두 블록에 각각 값을 넣으십시오. 실제로 이 실수가 한 번 있었습니다.

6. 클라이언트 컴포넌트에서 서버 전용 모듈 부르기

"use client" 가 있는 파일에서 `lib/prisma`, `lib/auth`, `lib/dal` 을 import하면 빌드가 실패합니다.

필요한 값은 서버 컴포넌트에서 **props로 내려주십시오**. `app/layout.tsx` 가 `Nav` 에 `user` 를 넘기는 방식이 그 예입니다.

7 막혔을 때: 오류 사전

오류 메시지	원인과 해결
Property 'session' does not exist on type 'PrismaClient'	스키마를 바꾸고 타입을 다시 만들지 않음 → <code>npx prisma generate</code>
You're importing a component that needs next/headers	클라이언트 컴포넌트에서 서버 전용 모듈을 부름 (함정 6 참고)
Environment variable not found: DATABASE_URL	<code>.env</code> 파일이 없거나 값이 비어 있음. Vercel이라면 환경변수를 바꾼 뒤 재배포 했는지 확인
Calling setState synchronously within an effect	린트 경고. <code>useEffect</code> 안에서 상태를 바꾸는 대신 렌더 중에 조정하십시오. <code>use-task-actions.ts</code> 에 실제 예가 있습니다
Hydration failed	서버가 그린 HTML과 브라우저가 그린 것이 다름. 보통 렌더 중에 <code>new Date()</code> 나 난수를 쓴 경우입니다
날짜가 하루씩 밀림	거의 항상 함정 2입니다. 직접 계산한 곳을 찾으십시오
화면이 안 바뀜 (DB는 바뀜)	수정 후 <code>router.refresh()</code> 를 부르지 않음
포트 3000이 이미 사용 중	이전 개발 서버가 살아 있음. 종료하거나 <code>npm run dev -- --port 3001</code>

막혔을 때 가장 좋은 도구는 git입니다

이 저장소의 커밋 메시지는 "무엇을 왜 고쳤고 어떻게 확인했는지"를 담고 있습니다. 특정 코드가 왜 그 모양인지 궁금하면 `git log -p --follow <파일이름>` 을 실행해 보십시오. 문서보다 정확합니다.

고치다 망가뜨렸다면 `git diff` 로 무엇을 바꿨는지 보고, `git checkout -- <파일>` 로 되돌릴 수 있습니다.

8 용어 사전

용어	뜻
App Router	Next.js에서 폴더 구조가 곧 주소가 되는 방식. <code>app/week/page.tsx</code> → <code>/week</code>
서버 컴포넌트	서버에서 실행되어 완성된 HTML을 보내는 컴포넌트. 기본값
클라이언트 컴포넌트	<code>"use client"</code> 가 붙은, 브라우저에서 실행되는 컴포넌트
Server Action	폼 제출을 서버 함수로 직접 처리하는 방식. <code>app/actions/auth.ts</code>
Proxy	모든 요청이 먼저 지나가는 문지기. Next.js 16에서 <code>middleware</code> 가 이 이름으로 바뀜
Prisma	데이터베이스를 JavaScript 함수로 다루게 해 주는 도구
마이그레이션	데이터베이스 구조 변경 이력. <code>prisma/migrations/</code>
스키마	데이터베이스 설계도. <code>prisma/schema.prisma</code>
DAL	Data Access Layer. 로그인한 사용자를 알아내는 단일 창구. <code>lib/dal.ts</code>
세션	로그인 상태. 쿠키에 든 토큰과 DB의 <code>Session</code> 행이 짝
IDOR	id만 바뀌어서 남의 데이터에 접근하는 취약점. 소유자 조건으로 막음
낙관적 UI	서버 응답을 기다리지 않고 화면을 먼저 바꾸는 방식
Tailwind	클래스 이름으로 CSS를 쓰는 도구
cascade	부모가 삭제되면 자식도 함께 삭제되는 설정. 사용자를 지우면 그 사람의 할 일도 사라짐

막히면 순서대로 시도하십시오. ① 오류 메시지를 끝까지 읽는다 → ② 7장 오류 사전을 본다 → ③ `git log` 로 그 파일의 이력을 본다 → ④ `npx prisma studio` 로 데이터가 실제로 어떤지 확인한다. 대부분은 여기서 풀립니다.